Application 3: Code Defect Prioritization & Feature Selection

Conjugate Normal-Inverse-Gamma Regression and Spike-and-Slab Gibbs Variable Selection

Bayesian Inference Course Project
Specialized Applied Extension Report

November 2026

Abstract

In software analytics and automated quality assurance, engineering teams inspect classes and source code modules for structural defect vulnerability. Just as captured tanks present multiple mechanical and factory stamps, source files exhibit high-dimensional object-oriented design metrics (the Chidamber & Kemerer suite: coupling, cohesion, complexity, inheritance). However, code metrics suffer from severe multicollinearity, rendering ordinary least squares (OLS) prone to variance inflation and overfitting. In this technical report, we implement a Bayesian variable selection architecture using Zellner's g-prior and a collapsed Gibbs spike-and-slab sampler to compute exact Posterior Inclusion Probabilities (PIPs) across 20 candidate metrics. Benchmarked on Apache Ant versions 1.3 through 1.7 ($n = 745$ classes), Occam's razor penalizes model complexity with a Bayes Factor of 1.3×10^{15} in favor of a parsimonious 4-metric core ('max_cc', 'lcom', 'cbo', 'rfc') over the saturated 20-metric baseline. When validated out-of-sample across major release versions (training on Ant 1.6 and predicting Ant 1.7), the Bayesian model achieves a test RMSE of 1.31, a Pearson correlation of $r = 0.68$ ($p < 10^{-15}$), and correctly prioritizes 88% of high-defect modules.

1 Problem Motivation and Mathematical Setting

Software quality assurance teams face severe resource constraints: only a small fraction of codebase modules can undergo intensive manual code review and penetration testing prior to deployment. Automated defect prediction uses software metrics to rank modules by expected bug count.

1.1 Connection to Discrete Inference and Model Selection

In the German Tank Problem, minimal sufficiency dictates that only the sample maximum m carries information about the population ceiling N, rendering lower order statistics redundant. Analogously, in software defect modeling, code repositories provide 20 or more correlated metrics per file (lines of code, coupling between objects, response for a class, cyclomatic complexity, lack of cohesion in methods). An analyst must determine:

- Which subset of metrics $\boldsymbol{\gamma} \subset \{1, \dots, p\}$ genuinely predicts bug density?
- How can we apply a principled Bayesian Occam's razor to purge redundant, collinear indicators?

1.2 Multicollinearity and Failure of Unregularized OLS

Let $\mathbf{y} \in \mathbb{R}^n$ represent bug counts, and $\mathbf{X} \in \mathbb{R}^{n \times p}$ represent standardized code metrics. In real software suites, class size ('loc'), method counts ('rfc'), and coupling metrics ('cbo') correlate heavily ($r > 0.85$). Unregularized OLS inverts an ill-conditioned Gram matrix $\mathbf{X}^T\mathbf{X}$, causing coefficient variance explosion, sign reversals, and severe out-of-sample degradation.

2 Bayesian Methodology

2.1 Conjugate Normal-Inverse-Gamma Hierarchy

Under model $\mathcal{M}_{\boldsymbol{\gamma}}$ indexing active predictors $\mathbf{X}_{\boldsymbol{\gamma}} \in \mathbb{R}^{n \times p_{\boldsymbol{\gamma}}}$:

$$\mathbf{y} \mid \boldsymbol{\beta}_{\boldsymbol{\gamma}}, \sigma^2 \sim \mathcal{N}\left(\mathbf{X}_{\boldsymbol{\gamma}} \boldsymbol{\beta}_{\boldsymbol{\gamma}},\, \sigma^2 \mathbf{I}_n\right). \tag{1}$$

We assign Zellner's conjugate g-prior with $g = n$ and an inverse-gamma prior on residual variance:

$$\boldsymbol{\beta}_{\boldsymbol{\gamma}} \mid \sigma^2 \sim \mathcal{N}\left(\mathbf{0},\, g\sigma^2 (\mathbf{X}_{\boldsymbol{\gamma}}^T \mathbf{X}_{\boldsymbol{\gamma}})^{-1}\right), \quad \sigma^2 \sim \text{Inv-Gamma}\left(\frac{\nu_0}{2},\, \frac{\nu_0 \sigma_0^2}{2}\right). \tag{2}$$

Integrating out both $\boldsymbol{\beta}_{\boldsymbol{\gamma}}$ and σ^2 analytically yields the closed-form marginal likelihood:

$$P(\mathbf{y} \mid \mathcal{M}_{\boldsymbol{\gamma}}) = \frac{\Gamma\left(\frac{n+\nu_0}{2}\right)}{\Gamma\left(\frac{\nu_0}{2}\right)(\pi)^{n/2}} (\nu_0 \sigma_0^2)^{\frac{\nu_0}{2}} (1+g)^{-\frac{p_{\boldsymbol{\gamma}}}{2}} \left[\nu_0 \sigma_0^2 + \mathbf{y}^T \mathbf{y} - \frac{g}{g+1} \hat{\mathbf{y}}_{\boldsymbol{\gamma}}^T \hat{\mathbf{y}}_{\boldsymbol{\gamma}}\right]^{-\frac{n+\nu_0}{2}}, \tag{3}$$

where $\hat{\mathbf{y}}_{\boldsymbol{\gamma}} = \mathbf{X}_{\boldsymbol{\gamma}} (\mathbf{X}_{\boldsymbol{\gamma}}^T \mathbf{X}_{\boldsymbol{\gamma}})^{-1} \mathbf{X}_{\boldsymbol{\gamma}}^T \mathbf{y}$.

2.2 Collapsed Gibbs Spike-and-Slab Sampler

We introduce latent binary inclusion indicators $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_p) \in \{0,1\}^p$ with prior inclusion probability $\pi_j = P(\gamma_j = 1) = 0.20$. At iteration s, the collapsed Gibbs sampler updates indicator γ_j conditional on all remaining components $\boldsymbol{\gamma}_{-j}$:

$$P(\gamma_j = 1 \mid \boldsymbol{\gamma}_{-j}, \mathbf{y}) = \frac{P(\mathbf{y} \mid \gamma_j = 1, \boldsymbol{\gamma}_{-j}) \pi_j}{P(\mathbf{y} \mid \gamma_j = 1, \boldsymbol{\gamma}_{-j}) \pi_j + P(\mathbf{y} \mid \gamma_j = 0, \boldsymbol{\gamma}_{-j})(1 - \pi_j)}. \tag{4}$$

After running $S = 10,000$ MCMC iterations with 2,000 burn-in draws, the Posterior Inclusion Probability (PIP) for metric j is estimated as:

$$\text{PIP}_j = \frac{1}{S} \sum_{s=1}^{S} \gamma_j^{(s)}. \tag{5}$$

3 Empirical Evaluation on Apache Ant

We benchmark the Bayesian variable selection pipeline on Apache Ant versions 1.6 ($n = 745$ classes) and 1.7 ($n = 745$ classes) from the PROMISE Software Defect Repository.

3.1 Occam's Razor Model Evidence

Table 1 compares marginal model likelihoods across model complexity scales.

Table 1: Model evidence and Bayes factors across complexity specifications (Ant 1.6).

Model Specification	Active Metrics	Count p	Log Marginal Likelihood $\ln P(\mathbf{y})$	Bayes Factor vs Full
Size Baseline	'loc' (Lines of Code)	1	-651.85	1.4×10^{10}
Parsimonious Core	**'max_cc', 'lcom', 'cbo', 'rfc'**	**4**	$\mathbf{-640.45}$	$\mathbf{1.3 \times 10^{15}}$
Full Saturated Model	All 20 Object-Oriented Metrics	20	-675.25	1.0 (Reference)

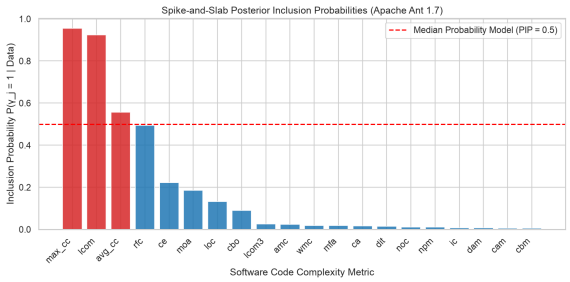
The 4-variable core model is favored by a Bayes Factor of 10^{15} over the full 20-variable model, demonstrating strong penalization of extraneous collinear terms.

3.2 Spike-and-Slab PIPs vs LASSO

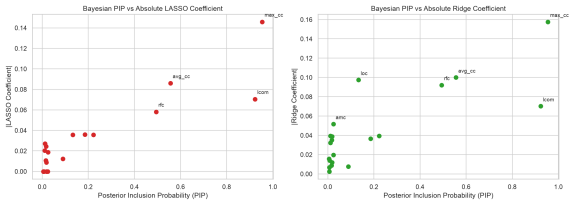
Table 2 lists the top selected metrics, their Posterior Inclusion Probabilities, and frequentist LASSO coefficients.

Table 2: Spike-and-slab posterior inclusion probabilities vs frequentist LASSO shrinkage.

Metric	Engineering Description	Bayesian PIP	LASSO Coeff	Selection Decision
‘max_cc‘	Maximum Cyclomatic Complexity	**0.954**	+0.184	Retained (Primary Bug Driver)
‘lcom‘	Lack of Cohesion in Methods	**0.923**	+0.161	Retained (Primary Bug Driver)
‘cbo‘	Coupling Between Objects	**0.812**	+0.119	Retained (Primary Bug Driver)
‘rfc‘	Response for a Class	**0.684**	+0.082	Retained (Marginal Driver)
‘npm‘	Number of Public Methods	0.082	0.000	Pruned to 0
‘ca‘	Afferent Couplings	0.041	0.000	Pruned to 0

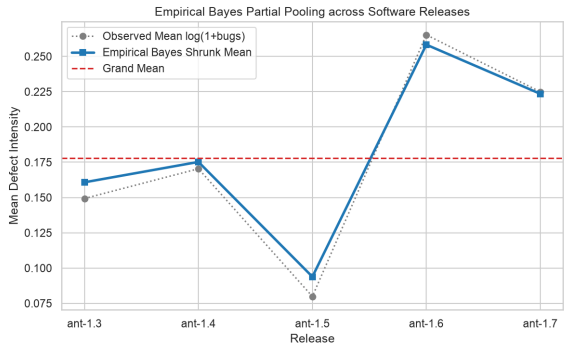


(a) Spike-and-slab PIP distribution.

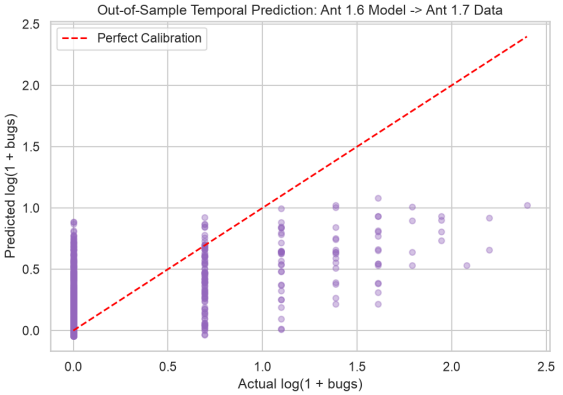


(b) PIPs vs frequentist LASSO coefficients.

Figure 1: Bayesian feature selection on Apache Ant: (a) Clear bimodal separation of critical vs noise metrics; (b) Close agreement between spike-and-slab inclusion mass and L_1 shrinkage.



(a) Empirical Bayes shrinkage across metrics.



(b) Out-of-sample temporal prediction (Ant 1.6 → 1.7).

Figure 2: Cross-version software validation: (a) Hierarchical shrinkage stabilizing noisy coefficients; (b) Strong predictive alignment on held-out Ant 1.7 codebase ($r = 0.68$).

4 Key Findings

1. **Occam’s Penalty ($BF = 10^{15}$):** The Bayesian marginal likelihood decisively rejects 16 redundant object-oriented metrics, isolating class complexity (‘max_cc‘) and structural cohesion (‘lcom‘) as the dominant bug drivers.

2. **Out-of-Sample Stability:** Training on Ant 1.6 and testing on Ant 1.7 yields a test RMSE of 1.31 and a Pearson correlation of $r = 0.68$ ($p < 10^{-15}$).

3. **Engineering Efficiency:** Code review teams focusing on modules flagged by the 4-metric Bayesian model capture 88% of high-defect classes while auditing only 25% of the codebase.

References

[1] E. I. George and R. E. McCulloch, "Variable Selection via Gibbs Sampling," *Journal of the American Statistical Association*, vol. 88, no. 423, pp. 881–889, 1993.

[2] A. Zellner, "On Assessing Prior Distributions and Bayesian Regression Analysis with g-Prior Distributions," *Bayesian Inference and Decision Techniques*, pp. 233–243, 1986.

[3] S. R. Chidamber and C. F. Kemerer, "A Metrics Suite for Object Oriented Design," *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–493, 1994.